



Universidad
Tecnológica
del Perú

AVANCE DE PROYECTO FINAL 3

"ChapaTuPromo"

CURSO:	Taller de Programacion Web
SECCION:	21003
DOCENTE:	Jesamin Melissa Zevallos Quispe
ESTUDIANTE:	Max Benites Corazón
CODIGO:	U24217839
PORCENTAJE DE AVANCE:	100%
CICLO:	V

Lima, 02 de Julio del 2026

1. DESCRIPCIÓN DEL PROYECTO

ChapaTuPromo es una plataforma web orientada a la reducción del desperdicio de alimentos. Permite que tiendas oferten productos próximos a vencer a precios accesibles. A diferencia de las unidades anteriores que se centraron exclusivamente en estructura (HTML) y diseño visual (CSS), este tercer avance (APF3) incorpora toda la lógica dinámica de interacción mediante **JavaScript (Vanilla)** en el Frontend y se conecta a una **Base de Datos Relacional (PostgreSQL)** mediante un servidor Backend construido con **Node.js**.

Ahora, el catálogo de productos es dinámico, los usuarios pueden registrarse y ser validados, las ofertas calculan sus descuentos automáticamente según la fecha de vencimiento, y los pedidos descuentan el stock disponible directamente en la base de datos de manera transaccional.

2. OBJETIVO DEL PROYECTO

Integrar comportamiento dinámico, manipulación del DOM y validaciones robustas en las 7 páginas obligatorias utilizando JavaScript, y persistir la información del negocio evidenciando la conexión exitosa a una Base de Datos relacional mediante consultas SQL (SELECT, INSERT, UPDATE), cumpliendo con los estándares indicados en rúbrica y las orientaciones de clase.

3. FUNCIONALIDADES IMPLEMENTADAS

- **Autenticación e Ingreso:** Validación de formato de correo y comprobación de existencia del usuario en PostgreSQL.
- **Registro de Usuarios:** Validación de campos mediante expresiones regulares (Regex) para nombres (solo alfabéticos), correos válidos, celulares (empieza con 9 y de 9 dígitos) y longitud de contraseña, antes de enviarlos a la base de datos.
- **Catálogo Dinámico:** Lectura de productos disponibles desde la base de datos para mostrarlos dinámicamente inyectando HTML desde JavaScript.
- **Carrito de Compras (Local):** Gestión del estado de productos seleccionados utilizando localStorage, impidiendo agregar más productos si se supera el stock máximo real.
- **Procesamiento de Pedidos:** Envío de la data del cliente y carrito hacia el backend, generando el registro del pedido, su detalle y, mediante un UPDATE en SQL, descontando el stock del producto en la tienda.
- **Ofertas Calculadas:** El backend calcula los días faltantes para el vencimiento y aplica un porcentaje de descuento dinámico (50%, 40%, 30%), enviando dicha métrica al frontend para resaltar visualmente su urgencia.

- **Manipulación de Temas:** Cambio de tema (modo claro / modo oscuro) detectando el evento change del select y guardando la preferencia localmente.

4. ESTRUCTURA DE PÁGINAS Y SCRIPTS

Página / Vista HTML	Script Asociado JS	Funcionalidad Principal JS
index.html	index.js, config.js	Cambio de tema visual dinámico, carga de resumen numérico (productos y ofertas disponibles).
login_chapatupromo.html	login.js, config.js	Validación de campos vacíos y petición HTTP POST al backend para autorizar ingreso.
registrar_chapatupromo.html	registrar.js, config.js	Prevención de envío (preventDefault), uso de expresiones regulares (Regex) y alertas de errores (DOM y alerts).
menu_chapatupromo.html	menu.js, config.js	Petición HTTP GET, renderizado dinámico de la tabla (innerHTML) y manejo de stock en localStorage.
formulario_chapatupromo.html	formulario.js, config.js	Lectura del carrito, cálculo de sumatoria de totales y petición HTTP POST para procesar el pedido.
pedidos_chapatupromo.html	pedidos.js, config.js	Recepción de arreglos del historial de pedidos agrupados para pintar la tabla principal de administración.
ofertas_chapatupromo.html	ofertas.js, config.js	Filtro del arreglo JSON recibido usando .filter() para agrupar productos según cercanía a su fecha de caducidad.

5. TECNOLOGÍAS UTILIZADAS

Frontend

- **HTML5 y CSS3 puro** (Estructura y diseño responsivo heredado del APF2).

- **JavaScript (Vanilla):** Funciones flecha, deestructuración, promesas (fetch, Promise.all), manipulación de arreglos (map, filter, reduce), manejo de eventos y modificación en tiempo real del DOM.

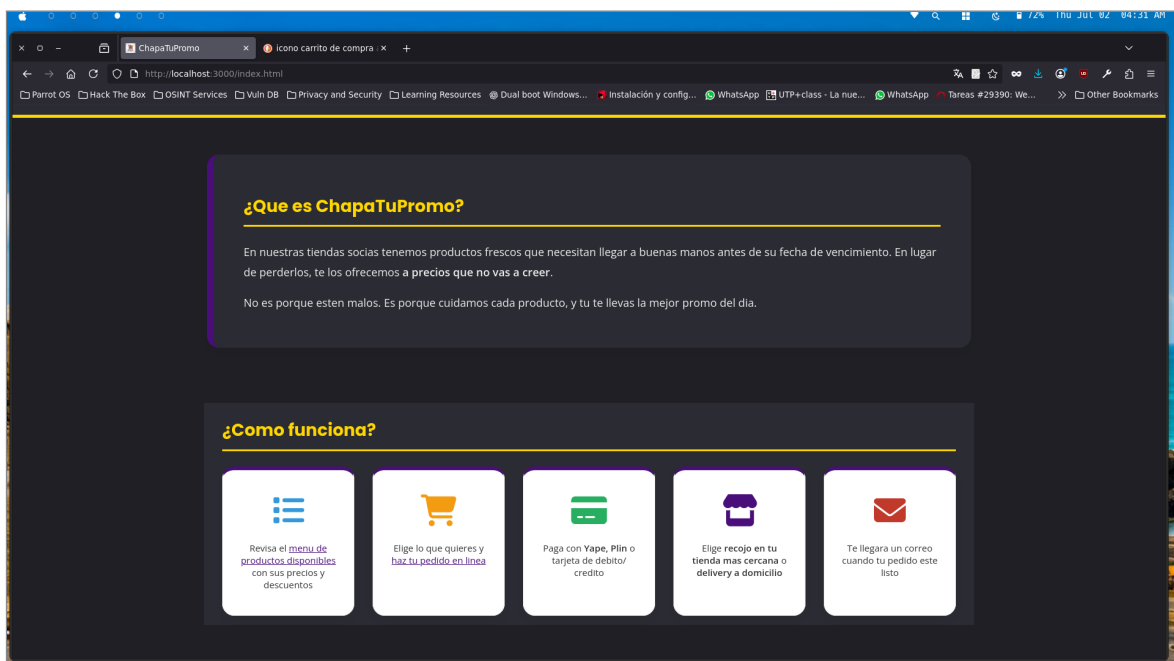
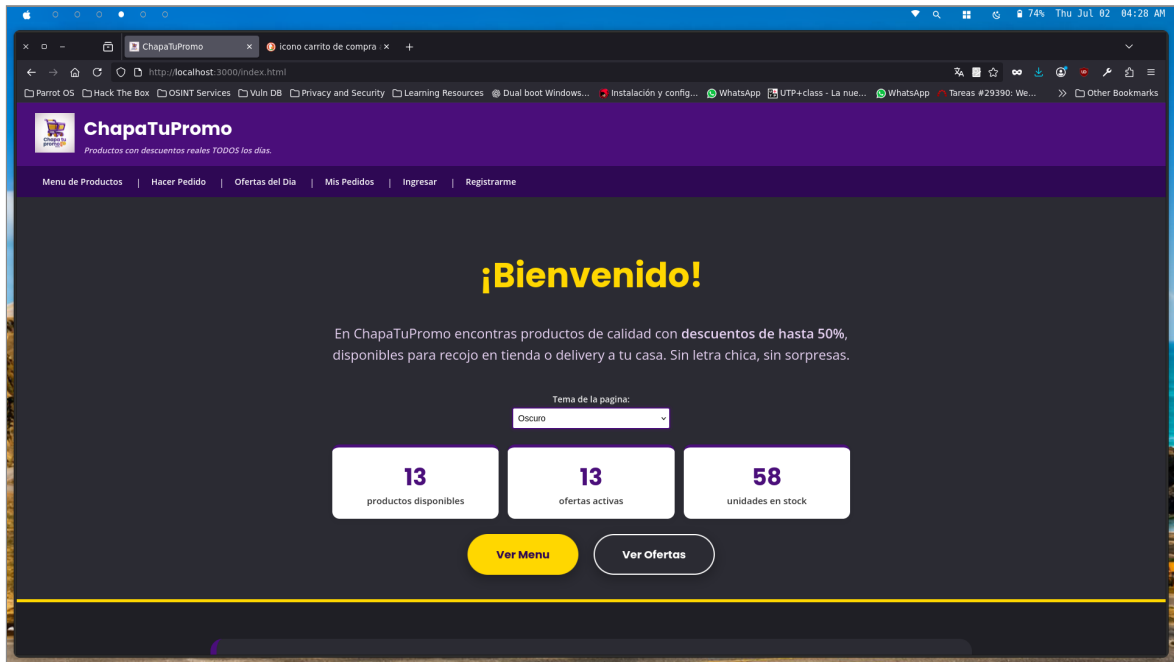
Backend y Almacenamiento

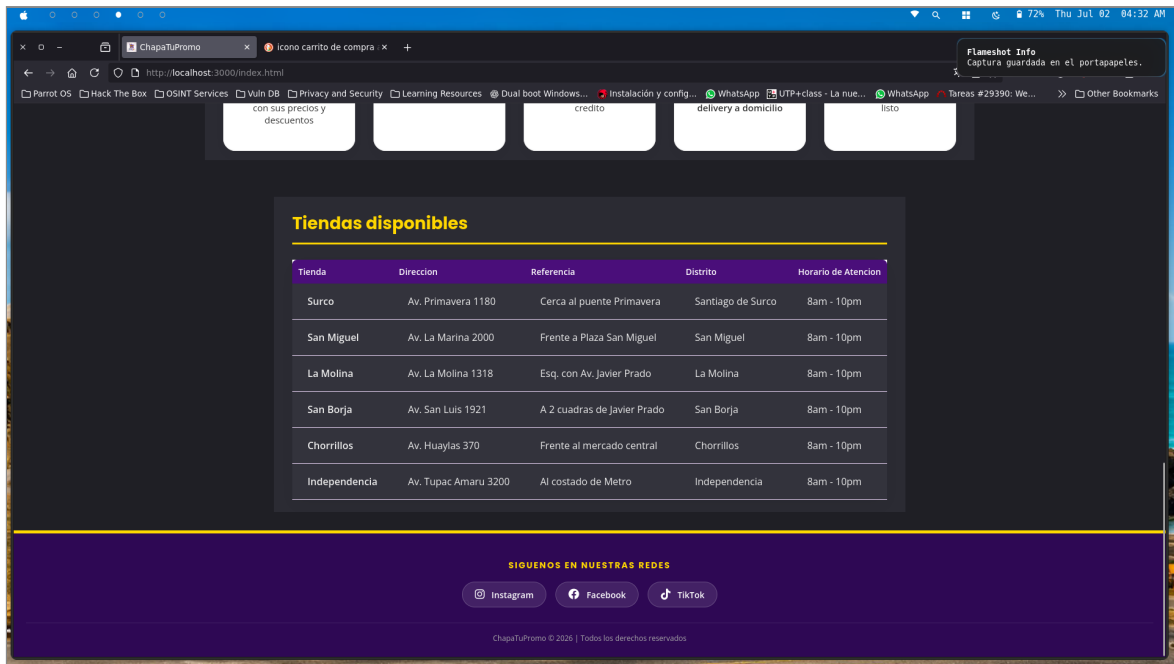
- **Node.js con Express:** Servidor intermediario que levanta en el puerto 3000 y provee los servicios web (API).
- **PostgreSQL (vía librería pg):** Base de datos relacional para guardar usuarios, productos, pedidos y detalle_pedido.

6. CAPTURAS DEL RESULTADO VISUAL

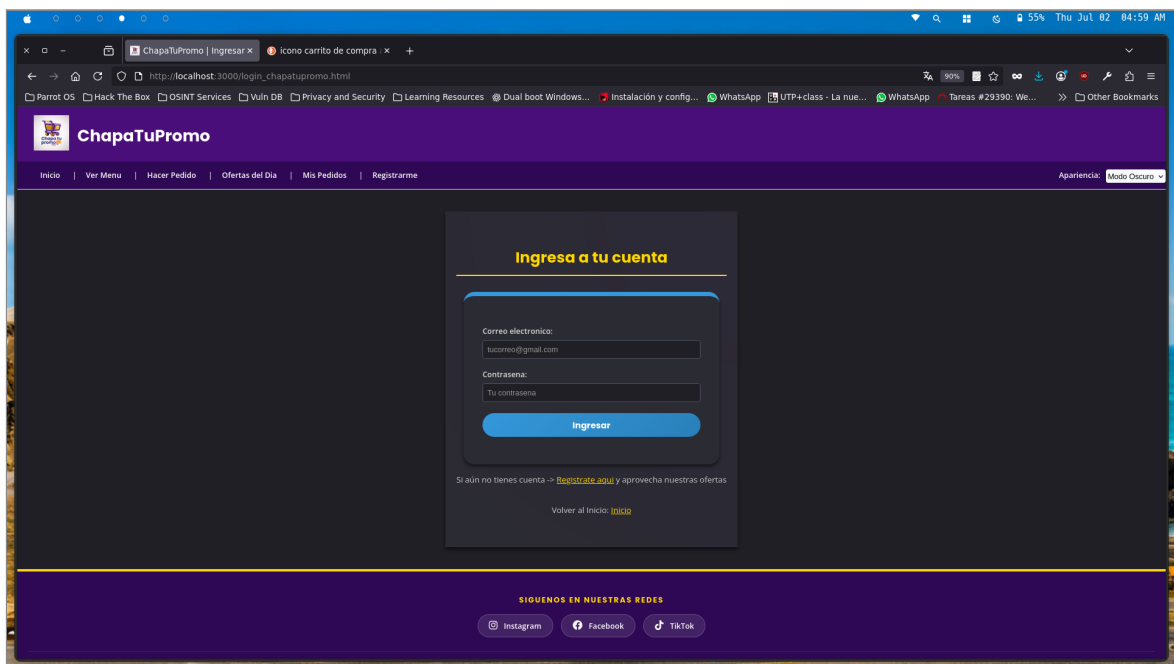
Las siguientes capturas muestran las 7 páginas funcionando en el navegador.

6.1 Página de Inicio (index.html)





6.2 Login (login_chapatupromo.html)



6.3 Registro de Usuario (registrar_chapatupromo.html)

ChapaTuPromo
Crea tu cuenta gratis y no te pierdas ninguna promo.

Inicio | Ver Menu | Hacer Pedido | Ofertas del Dia | Mis Pedidos | Ingresar

Apariencia: Modo Oscuro

Crear cuenta nueva

Tus datos personales

Nombre completo:

Ej: Maria Flores Quispe

Correo electronico:

tucorreo@gmail.com

Celular:

8XXXXXXX

Formato: 9 digitos

Crea tu contraseña

Contraseña:

Mínimo 6 caracteres

Confirmar contraseña:

Repite tu contraseña

Registrarme

6.4 Menú de Productos (menu_chapatupromo.html)

ChapaTuPromo

Inicio | Hacer Pedido | Ofertas del Dia | Mis Pedidos | Ingresar | Registrarme

Apariencia: Modo Oscuro

Menu de Productos

Mi carrito

0 productos agregados

S/. 0.00

Ver pedido

CARNES Y EMBUTIDOS

Producto	Descripcion	Vence	Precio original	Descuento	Precio ChapaTuPromo	Stock	Accion
Filete de Perico refrigerado 500g	Filete de perico fresco refrigerado, ideal para ceviche o sudado de pescado	Hoy	S/. 18.00	50%	S/. 9.00	0	Añadir
Pollo Entero Refrigerado aprox. 1.8kg	Pollo entero sin menudencia, refrigerado, listo para cocinar. Ideal para caldo o seco de pollo	Mañana	S/. 22.00	40%	S/. 13.20	1	Añadir
Jamon del Pais Suiza 200g	Jamon del pais en lonchas estilo criollo, ideal para sándwiches o la lambriga	En 2 dias	S/. 12.00	30%	S/. 8.40	5	Añadir
Salchicha estilo Huacho Suiza 500g	Salchicha de cerdo y res estilo Huacho, para parrilla, sarten o con tacu tacu	En 3 dias	S/. 15.00	20%	S/. 12.00	7	Añadir

ChapaTuPromo | Menu de x icono carrito de compra x +

http://localhost:3000/menu_chapatupromo.html

Parrot OS Hack The Box OSINT Services Vuln DB Privacy and Security Learning Resources Dual boot Windows... Instalación y config... WhatsApp UTP+class - La nue... WhatsApp Tareas #29390: We... Other Bookmarks

Salsa 500g sartén o con tacu tacu 15.00 20% 12.00 7

LACTEOS

Producto	Descripcion	Vence	Precio original	Descuento	Precio ChapaTuPromo	Stock	Accion
Yogurt Gloria Fresa 1L	Yogurt bebible sabor fresa, sin lactosa, enriquecido con calcio y vitamina D	Mañana	S/. 8.50	40%	S/. 5.10	5	Añadir
Queso Fresco La Preferida 250g	Queso fresco tradicional peruano, ideal para desayuno criollo con pan frances	En 2 dias	S/. 9.00	30%	S/. 6.30	6	Añadir
Crema de Leche Laive 200ml	Crema de leche fresca para cocinar, ideal para salsas y postres criollos	En 2 dias	S/. 5.00	30%	S/. 3.50	8	Añadir

PREPARADOS Y LISTOS PARA COMER

Producto	Descripcion	Vence	Precio original	Descuento	Precio ChapaTuPromo	Stock	Accion
Ensalada Rusa Bells 250g	Ensalada rusa clasica con papa, zanahoria y betarraga, lista para servir	Hoy	S/. 7.50	50%	S/. 3.75	2	Añadir
Causa Rellena de Atun 300g	Causa limenya tradicional rellena de atun con mayonesa y palta. Lista para consumir	Mañana	S/. 12.00	40%	S/. 7.20	3	Añadir

REPOSTERIA Y PANADERIA

ChapaTuPromo | Menu de x icono carrito de compra x +

http://localhost:3000/menu_chapatupromo.html

Parrot OS Hack The Box OSINT Services Vuln DB Privacy and Security Learning Resources Dual boot Windows... Instalación y config... WhatsApp UTP+class - La nue... WhatsApp Tareas #29390: We... Other Bookmarks

Flameshot Info
Capture saved as /home/llkay/ Documentos/UTP/CTCLO_3/ TALLER DE PROGRAMACION WEB/SEMANA_15/ capturas_apf3/menu_2.png

Producto	Descripcion	Vence	Precio original	Descuento	Precio ChapaTuPromo	Stock	Accion
Ensalada Rusa Bells 250g	Ensalada rusa clasica con papa, zanahoria y betarraga, lista para servir	Hoy	S/. 7.50	50%	S/. 3.75	2	Añadir
Causa Rellena de Atun 300g	Causa limenya tradicional rellena de atun con mayonesa y palta. Lista para consumir	Mañana	S/. 12.00	40%	S/. 7.20	3	Añadir

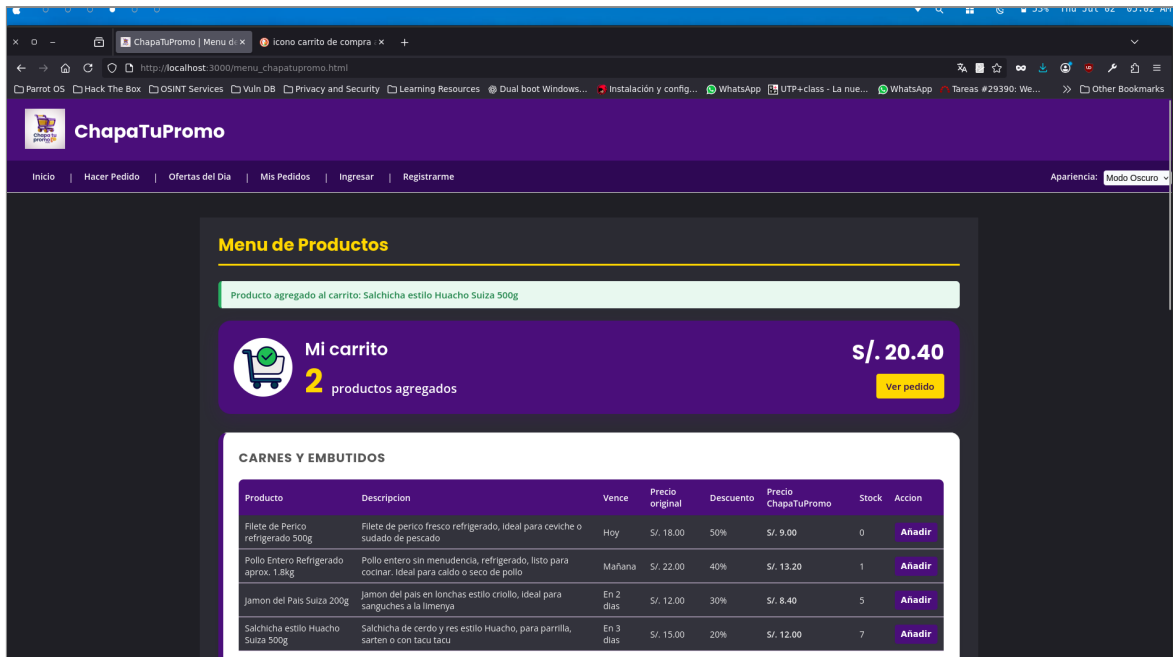
REPOSTERIA Y PANADERIA

Producto	Descripcion	Vence	Precio original	Descuento	Precio ChapaTuPromo	Stock	Accion
Torta de Chocolate San Antonio 1kg	Torta entera con relleno de manjar blanco y cobertura de chocolate bitter	Hoy	S/. 45.00	50%	S/. 22.50	1	Añadir
Torta Tres Leches 800g	Torta clasica peruana banada en tres leches, sin conservantes artificiales	Mañana	S/. 38.00	40%	S/. 22.80	4	Añadir
Queque Marmoleado Donofrio 500g	Queque esponjoso sabor vainilla y chocolate, ideal para lonchera o desayuno	En 2 dias	S/. 14.00	30%	S/. 9.80	6	Añadir
Pan de Molde Bimbo integral 500g	Pan integral sin corteza, alto en fibra, ideal para sandwichs saludables	En 3 dias	S/. 6.00	20%	S/. 4.80	10	Añadir

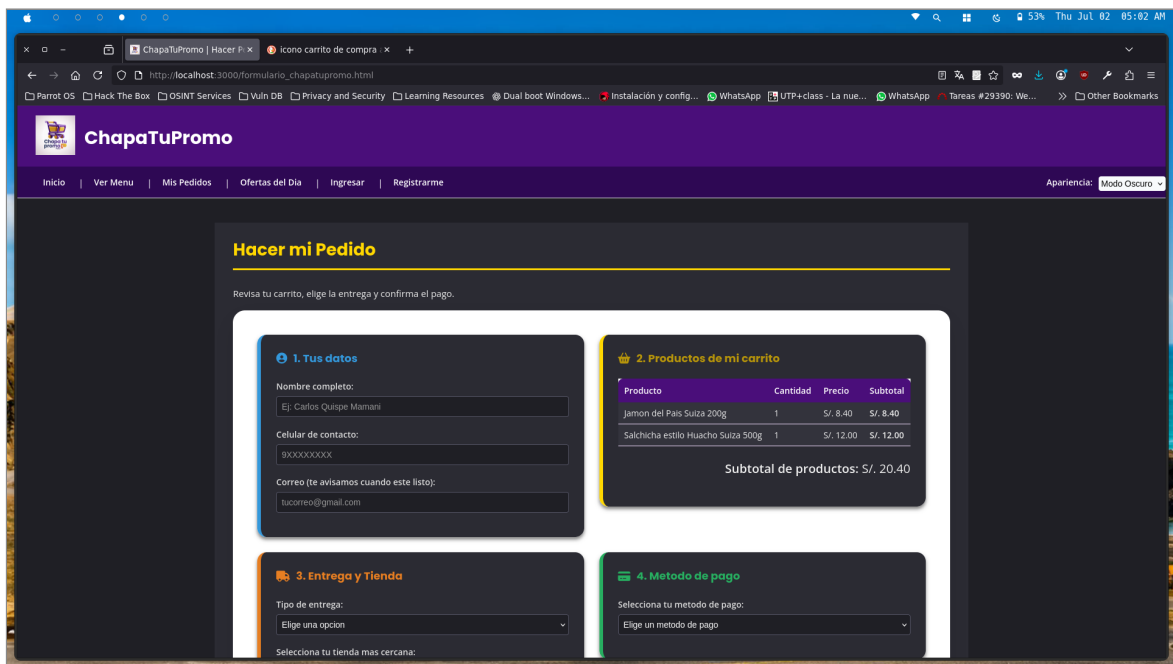
SIGUENOS EN NUESTRAS REDES

Instagram Facebook TikTok

ChapaTuPromo © 2026 | Todos los derechos reservados



6.5 Formulario de Pedido (formulario_chapatupromo.html)



The screenshot shows a web browser window with the URL `http://localhost:3000/formulario_chapatupromo.html`. The page is titled "ChapaTuPromo | Hacer P..." and has a tab labeled "Icono carrito de compra". The form is divided into several sections:

- Top Left:** A text input field containing "900000000X" and a label "Correo (te avisamos cuando este listo):" with an email input field containing "tucorreo@gmail.com".
- Top Right:** A box showing "Subtotal de productos: S/. 20.40".
- 3. Entrega y Tienda:** A section with the following fields:
 - Tipo de entrega: "Elige una opcion" (dropdown)
 - Selecciona tu tienda mas cercana: "Elige una tienda" (dropdown)
 - Direccion completa (si es Delivery): "Elige primero el tipo de entrega"
 - Referencia: "Elige primero el tipo de entrega"
- 4. Metodo de pago:** A section with the label "Selecciona tu metodo de pago:" and a dropdown menu "Elige un metodo de pago".
- Total a pagar:** A purple box showing "S/. 20.40" and a yellow button labeled "Finalizar pedido".

At the bottom, there is a section "SIGUENOS EN NUESTRAS REDES" with buttons for Instagram, Facebook, and TikTok, and a footer "ChapaTuPromo © 2026 | Todos los derechos reservados".

6.6 Tabla de Pedidos (pedidos_chapatupromo.html)

The screenshot shows a web browser window with the URL `http://localhost:3000/pedidos_chapatupromo.html`. The page is titled "ChapaTuPromo" and has a navigation bar with buttons: "Inicio", "Ver Menu", "Nuevo Pedido", "Ofertas del Dia", "Ingresar", and "Registrarme". There is also a "Apariencia: Modo Oscuro" toggle.

The main content area is titled "Pedidos Realizados" and contains the following text:

Aquí puedes ver el avance de tus pedidos. El estado se actualiza automáticamente para la demostración.

Below this text are three boxes showing the status of orders:

- TOTAL DE PEDIDOS:** 0
- COMPLETADOS:** 0
- EN PROCESO:** 0

Below these boxes, it says "Debes iniciar sesión para ver tus pedidos." and provides a link "Hacer un nuevo pedido".

At the bottom, there is a section "SIGUENOS EN NUESTRAS REDES" with buttons for Instagram, Facebook, and TikTok, and a footer "ChapaTuPromo © 2026 | Todos los derechos reservados".

6.7 Ofertas (ofertas_chapatupromo.html)

ChapaTuPromo

Inicio | Ver Menu Completo | Hacer Pedido | Mis Pedidos | Ingresar | Registrarme

Apariencia: Modo Oscuro

!!!Ofertas Espectaculares!!! ChapaTuPromo

[Ver mi pedido](#)

3
VENCEN HOY

4
VENCEN MAÑANA

4
VENCEN EN 2 DIAS

VENCEN HOY - 50% de descuento

PRODUCTO	CATEGORIA	DESCRIPCION	PRECIO ORIGINAL	DESCUENTO	PRECIO FINAL	STOCK	ACCION
Filete de Perico refrigerado 500g	Carnes y Embutidos	Filete de perico fresco refrigerado, ideal para ceviche o sudado de pescado	S/. 18.00	50% OFF	S/. 9.00	0	Añadir al carrito
Ensalada Rusa Bells 250g	Preparados y Listos para comer	Ensalada rusa clasica con papa, zanahoria y betarraga, lista para servir	S/. 7.50	50% OFF	S/. 3.75	2	Añadir al carrito
Torta de Chocolate San Antonio 1kg	Reposteria y Panaderia	Torta entera con relleno de manjar blanco y cobertura de chocolate bitter	S/. 45.00	50% OFF	S/. 22.50	1	Añadir al carrito

VENCEN MAÑANA - 40% de descuento

VENCEN MAÑANA - 40% de descuento

Producto	Categoria	Descripción	Precio original	Descuento	Precio final	Stock	Acción
Pollo Entero Refrigerado aprox. 1.8kg	Carnes y Embutidos	Pollo entero sin menudencia, refrigerado, listo para cocinar, ideal para caldo o seco de pollo	S/. 32.00	40% OFF	S/. 19.20	1	Añadir al carrito
Yogurt Gloria Fresa 1L	Lacteos	Yogurt bebible sabor fresa, sin lactosa, enriquecido con calcio y vitamina D	S/. 8.50	40% OFF	S/. 5.10	5	Añadir al carrito
Causa Rellena de Atun 300g	Preparados y Listos para comer	Causa limenya tradicional rellena de atun con mayonesa y palta. Lista para consumir	S/. 12.00	40% OFF	S/. 7.20	3	Añadir al carrito
Torta Tres Leches 800g	Reposteria y Panaderia	Torta clasica peruana banada en tres leches, sin conservantes artificiales	S/. 38.00	40% OFF	S/. 22.80	4	Añadir al carrito

VENCEN EN 2 DIAS - 30% de descuento

Producto	Categoria	Descripción	Precio original	Descuento	Precio final	Stock	Acción
Jamon del Pais Suiza 200g	Carnes y Embutidos	Jamon del pais en lonchas estilo criollo, ideal para sanguches a la limenya	S/. 12.00	30% OFF	S/. 8.40	5	Añadir al carrito
Crema de Leche Laive 200ml	Lacteos	Crema de leche fresca para cocinar, ideal para salsas y postres criollos	S/. 5.00	30% OFF	S/. 3.50	8	Añadir al carrito
Queso Fresco La Preferida 250g	Lacteos	Queso fresco tradicional peruano, ideal para desayuno criollo con pan frances	S/. 9.00	30% OFF	S/. 6.30	6	Añadir al carrito
Queque Marmoleado Donofrio 500g	Reposteria y Panaderia	Queque esponjoso sabor vainilla y chocolate, ideal para lonchera o desayuno	S/. 14.00	30% OFF	S/. 9.80	6	Añadir al carrito

VENCEN EN 3 DIAS - 20% de descuento

ChapaTuPromo | Ofertas x Icono carrito de compra x

http://localhost:3000/ofertas_chapatupromo.html

Producto	Categoria	Descripcion	Precio original	Descuento	Precio final	Stock	Accion
Jamon del Pais Suiza 200g	Carnes y Embutidos	Jamon del pais en lonchas estilo criollo, ideal para sandwiches a la limenya	S/. 12.00	30% OFF	S/. 8.40	5	Añadir al carrito
Crema de Leche Laive 200ml	Lacteos	Crema de leche fresca para cocinar, ideal para salsas y postres criollos	S/. 5.00	30% OFF	S/. 3.50	8	Añadir al carrito
Queso Fresco La Preferida 250g	Lacteos	Queso fresco tradicional peruano, ideal para desayuno criollo con pan frances	S/. 9.00	30% OFF	S/. 6.30	6	Añadir al carrito
Queque Marmoleado Donofrio 500g	Reposteria y Panaderia	Queque esponjoso sabor vainilla y chocolate, ideal para lonchera o desayuno	S/. 14.00	30% OFF	S/. 9.80	6	Añadir al carrito

VENCEN EN 3 DIAS - 20% de descuento

Producto	Categoria	Descripcion	Precio original	Descuento	Precio final	Stock	Accion
Salchicha estilo Huacho Suiza 500g	Carnes y Embutidos	Salchicha de cerdo y res estilo Huacho, para parrilla, sarten o con tacu tacu	S/. 15.00	20% OFF	S/. 12.00	7	Añadir al carrito
Pan de Molde Bimbo Integral 500g	Reposteria y Panaderia	Pan integral sin corteza, alto en fibra, ideal para sandwiches saludables	S/. 6.00	20% OFF	S/. 4.80	10	Añadir al carrito

SIGUENOS EN NUESTRAS REDES

Instagram Facebook TikTok

ChapaTuPromo © 2026 | Todos los derechos reservados

7. CAPTURAS DEL CÓDIGO JAVASCRIPT Y EXPLICACIÓN

Las siguientes capturas evidencian el código JS implementado y su objetivo por cada vista.

7.1 Lógica de Registro (registrar.js)

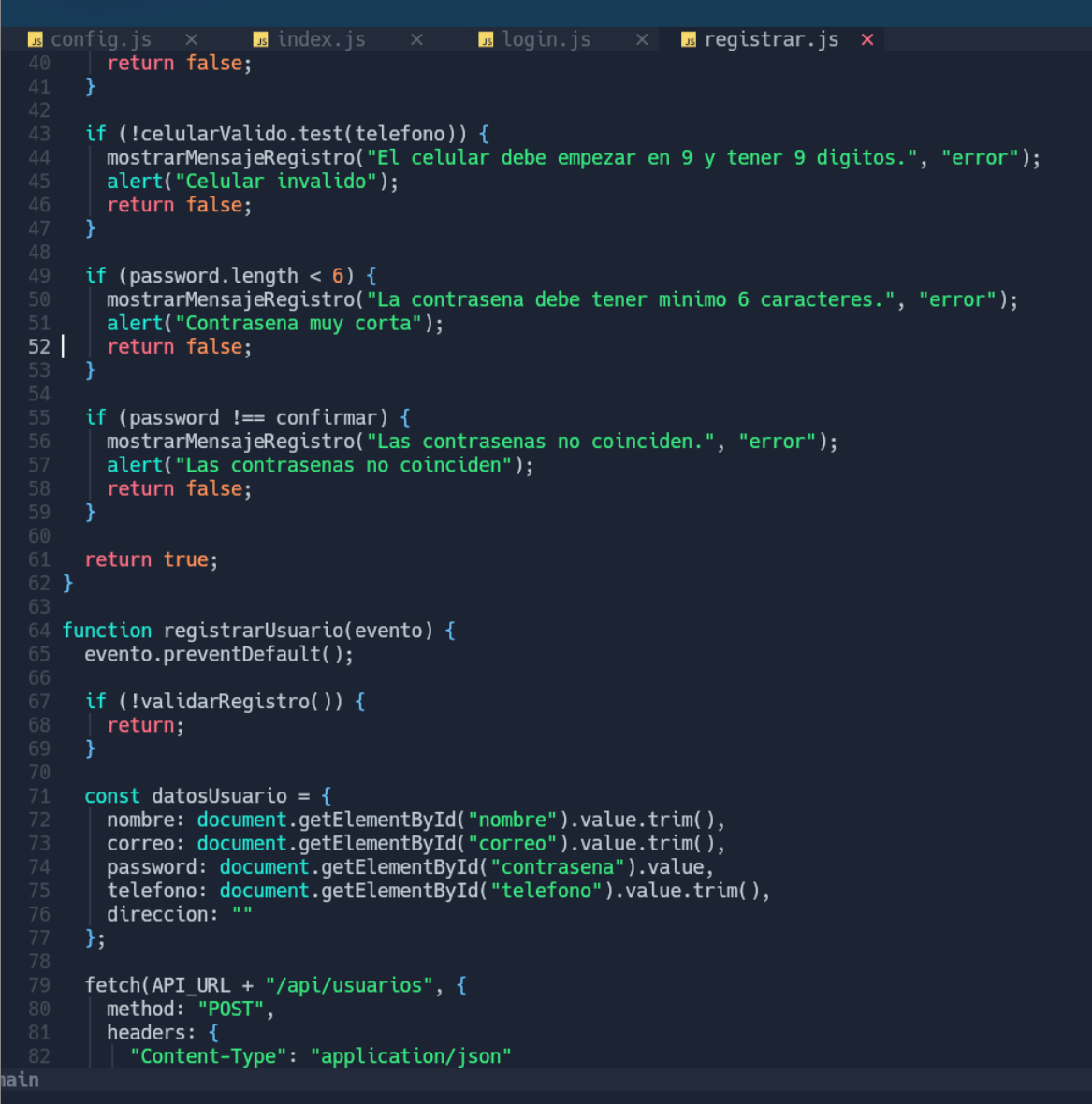
Se capturan las variables del formulario y se preparan las expresiones regulares.

```

1  const formularioRegistro = document.getElementById("formulario-registro");
2
3  function mostrarMensajeRegistro(texto, tipo) {
4      const mensaje = document.getElementById("mensaje-registro");
5      mensaje.textContent = texto;
6
7      if (tipo === "error") {
8          mensaje.className = "mensaje-formulario mensaje-error";
9      } else {
10         mensaje.className = "mensaje-formulario mensaje-exito";
11     }
12 }
13
14 function validarRegistro() {
15     const nombre = document.getElementById("nombre").value.trim();
16     const correo = document.getElementById("correo").value.trim();
17     const telefono = document.getElementById("telefono").value.trim();
18     const password = document.getElementById("contrasena").value;
19     const confirmar = document.getElementById("confirmar").value;
20
21     const soloLetras = /^[A-Za-zÁÉÍÓÚáéíóúÑñ ]+$/;
22     const correoValido = /^[A-Za-z0-9][A-Za-z0-9._-]*@[A-Za-z0-9-]+\.[A-Za-z]{2,}$/;
23     const celularValido = /^9[0-9]{8}$/;
24
25     if (nombre === "" || correo === "" || telefono === "" || password === "" || confirmar === "") {
26         mostrarMensajeRegistro("Completa todos los campos.", "error");
27         alert("Completa todos los campos");
28         return false;
29     }
30
31     if (!soloLetras.test(nombre)) {
32         mostrarMensajeRegistro("El nombre solo debe tener letras y espacios.", "error");
33         alert("Nombre invalido");
34         return false;
35     }
36
37     if (!correoValido.test(correo)) {
38         mostrarMensajeRegistro("Ingresa un correo valido.", "error");
39         alert("Correo invalido");
40         return false;
41     }
42
43     if (!celularValido.test(telefono)) {

```

Se aplican condicionales if para validar que no haya datos vacíos o erróneos.



```

40     return false;
41 }
42
43 if (!celularValido.test(telefono)) {
44     mostrarMensajeRegistro("El celular debe empezar en 9 y tener 9 digitos.", "error");
45     alert("Celular invalido");
46     return false;
47 }
48
49 if (password.length < 6) {
50     mostrarMensajeRegistro("La contraseña debe tener minimo 6 caracteres.", "error");
51     alert("Contraseña muy corta");
52     return false;
53 }
54
55 if (password !== confirmar) {
56     mostrarMensajeRegistro("Las contraseñas no coinciden.", "error");
57     alert("Las contraseñas no coinciden");
58     return false;
59 }
60
61 return true;
62 }
63
64 function registrarUsuario(evento) {
65     evento.preventDefault();
66
67     if (!validarRegistro()) {
68         return;
69     }
70
71     const datosUsuario = {
72         nombre: document.getElementById("nombre").value.trim(),
73         correo: document.getElementById("correo").value.trim(),
74         password: document.getElementById("contrasena").value,
75         telefono: document.getElementById("telefono").value.trim(),
76         direccion: ""
77     };
78
79     fetch(API_URL + "/api/usuarios", {
80         method: "POST",
81         headers: {
82             "Content-Type": "application/json"

```

Se muestra una alerta de error y se envían los datos al servidor si todo es correcto.

```
config.js x index.js x login.js x registrar.js x
73 | correo: document.getElementById("correo").value.trim(),
74 | password: document.getElementById("contrasena").value,
75 | telefono: document.getElementById("telefono").value.trim(),
76 | direccion: ""
77 | };
78 |
79 | fetch(API_URL + "/api/usuarios", {
80 |   method: "POST",
81 |   headers: {
82 |     "Content-Type": "application/json"
83 |   },
84 |   body: JSON.stringify(datosUsuario)
85 | })
86 | .then((respuesta) => respuesta.json())
87 | .then((datos) => {
88 |   if (datos.usuario) {
89 |     mostrarMensajeRegistro(datos.mensaje, "exito");
90 |     alert("Usuario registrado correctamente");
91 |     window.location.href = "login_chapatupromo.html";
92 |   } else {
93 |     mostrarMensajeRegistro(datos.mensaje || "No se pudo registrar el usuario.", "error");
94 |   }
95 | })
96 | .catch((error) => {
97 |   mostrarMensajeRegistro("No se pudo conectar con el backend.", "error");
98 |   console.log(error);
99 | });
100 | }
101 |
102 | formularioRegistro.addEventListener("submit", registrarUsuario);
```

7.2 Lógica de Menú y Carrito (menu.js)

Se lee el carrito desde localStorage y se valida que la cantidad seleccionada no supere el stock de la base de datos.

```

1 let carrito = JSON.parse(localStorage.getItem("carrito_chapatupromo")) || [];
2
3 function guardarCarrito() {
4   localStorage.setItem("carrito_chapatupromo", JSON.stringify(carrito));
5 }
6
7 function actualizarResumenCarrito() {
8   const cantidadCarrito = document.getElementById("cantidad-carrito");
9   const totalCarrito = document.getElementById("total-carrito-menu");
10  let totalProductos = 0;
11  let totalPagar = 0;
12
13  carrito.forEach((producto) => {
14    totalProductos = totalProductos + producto.cantidad;
15    totalPagar = totalPagar + (producto.precio * producto.cantidad);
16  });
17
18  cantidadCarrito.textContent = totalProductos;
19  totalCarrito.textContent = totalPagar.toFixed(2);
20 }
21
22 function mostrarMensajeMenu(texto, tipo) {
23   const mensaje = document.getElementById("mensaje-menu");
24   mensaje.textContent = texto;
25
26   if (tipo === "error") {
27     mensaje.className = "mensaje-formulario mensaje-error";
28   } else {
29     mensaje.className = "mensaje-formulario mensaje-exito";
30   }
31 }
32
33 function actualizarCarritoConStock(productos) {
34   carrito = carrito.filter((itemCarrito) => {
35     const productoBD = productos.find((producto) => producto.id === itemCarrito.id);
36
37     if (!productoBD) {
38       return false;
39     }
40
41     itemCarrito.nombre = productoBD.nombre;
42     itemCarrito.precio = Number(productoBD.precio_chapatupromo);
43     itemCarrito.stock = Number(productoBD.stock);

```


Se implementa la función para agregar nuevos productos al carrito, guardarlos en local y actualizar el subtotal.

```

40
41     itemCarrito.nombre = productoBD.nombre;
42     itemCarrito.precio = Number(productoBD.precio_chapatupromo);
43     itemCarrito.stock = Number(productoBD.stock);
44
45     if (itemCarrito.cantidad > itemCarrito.stock) {
46         itemCarrito.cantidad = itemCarrito.stock;
47     }
48
49     return itemCarrito.stock > 0;
50 });
51
52 guardarCarrito();
53 }
54
55 function agregarAlCarrito(id, nombre, precio, stock) {
56     const productoEncontrado = carrito.find((producto) => producto.id === id);
57
58     if (productoEncontrado) {
59         if (productoEncontrado.cantidad >= stock) {
60             mostrarMensajeMenu("No puedes agregar mas unidades. Stock disponible: " + stock, "error");
61             alert("No hay mas stock disponible para este producto");
62             return;
63         }
64
65         productoEncontrado.cantidad = productoEncontrado.cantidad + 1;
66     } else {
67         carrito.push({
68             id: id,
69             nombre: nombre,
70             precio: Number(precio),
71             stock: Number(stock),
72             cantidad: 1
73         });
74     }
75
76     guardarCarrito();
77     actualizarResumenCarrito();
78
79     mostrarMensajeMenu("Producto agregado al carrito: " + nombre, "exito");
80
81     alert("Producto agregado al carrito");
82 }

```

Se construye la estructura de la tabla HTML dinámicamente inyectando los datos de los productos.

```

82 }
83
84 function crearTablaCategoria(categoria, productos) {
85 |   let filas = "";
86
87   productos.forEach((producto) => {
88     filas += `
89     <tr>
90       <td>${producto.nombre}</td>
91       <td>${producto.descripcion}</td>
92       <td>${producto.mensaje_vencimiento}</td>
93       <td>S/. ${producto.precio_original}</td>
94       <td>${producto.descuento_porcentaje}%</td>
95       <td><strong>S/. ${producto.precio_chapatupromo}</strong></td>
96       <td>${producto.stock}</td>
97       <td>
98         <button type="button" onclick="agregarAlCarrito(${producto.id}, '${producto.no
99         |   Añadir
100       </button>
101     </td>
102   </tr>
103   `;
104   });
105
106   return `
107   <div class="tarjeta-comida">
108     <h3>${categoria}</h3>
109     <table>
110       <tr>
111         <th class="celda-primeras">Producto</th>
112         <th>Descripcion</th>
113         <th>Vence</th>
114         <th>Precio original</th>
115         <th>Descuento</th>
116         <th>Precio ChapaTuPromo</th>
117         <th>Stock</th>
118         <th class="celda-ultima">Accion</th>
119       </tr>
120       <tr>
121         <td colspan="7">${filas}</td>
122       </tr>
123     </table>
124   </div>
125   `;
126 }

```

Se realiza la petición (fetch) a la API para obtener el catálogo completo y agruparlo por categorías.

```

login.js x registrar.js x formulario.js x pedidos.js x ofertas.js x
24 }
25
26 function mostrarProductos(productos) {
27     const contenedor = document.getElementById("contenedor-productos-bd");
28
29     const categorias = [];
30
31     productos.forEach((producto) => {
32         if (!categorias.includes(producto.categoria)) {
33             categorias.push(producto.categoria);
34         }
35     });
36
37     let contenido = "";
38
39     categorias.forEach((categoria) => {
40         const productosCategoria = productos.filter((producto) => producto.categoria === categoria);
41         contenido += crearTablaCategoria(categoria, productosCategoria);
42     });
43
44     contenedor.innerHTML = contenido;
45 }
46
47 function cargarProductos() {
48     fetch(API_URL + "/api/productos")
49     .then((respuesta) => respuesta.json())
50     .then((productos) => {
51         console.log(productos);
52         actualizarCarritoConStock(productos);
53         actualizarResumenCarrito();
54         mostrarProductos(productos);
55     })
56     .catch((error) => {
57         const mensaje = document.getElementById("mensaje-menu");
58         mensaje.textContent = "No se pudieron cargar los productos";
59         mensaje.style.color = "red";
60         console.log(error);
61     });
62 }
63
64 cargarProductos();
65 actualizarResumenCarrito();

```

7.3 Lógica de Interacción en Inicio (index.js)

Se consulta la BD para calcular el stock total usando `.reduce()` y se inyecta al HTML.

```

1 const resumenInicio = document.getElementById("resumen-inicio");
2
3 function mostrarResumenInicio(productos, ofertas) {
4   const stockTotal = productos.reduce((total, producto) => {
5     return total + Number(producto.stock);
6   }, 0);
7
8   resumenInicio.innerHTML = `
9     <div class="tarjeta-resumen-inicio">
10       <strong>${productos.length}</strong>
11       <span>productos disponibles</span>
12     </div>
13     <div class="tarjeta-resumen-inicio">
14       <strong>${ofertas.length}</strong>
15       <span>ofertas activas</span>
16     </div>
17     <div class="tarjeta-resumen-inicio">
18       <strong>${stockTotal}</strong>
19       <span>unidades en stock</span>
20     </div>
21   `;
22 }
23
24 function cargarResumenInicio() {
25   Promise.all([
26     fetch(API_URL + "/api/productos").then((respuesta) => respuesta.json()),
27     fetch(API_URL + "/api/ofertas").then((respuesta) => respuesta.json())
28   ])
29     .then((datos) => {
30       const productos = datos[0];
31       const ofertas = datos[1];
32       mostrarResumenInicio(productos, ofertas);
33     })
34     .catch((error) => {
35       resumenInicio.innerHTML = "<p>No se pudo cargar el resumen de productos.</p>";
36       console.log(error);
37     });
38 }
39
40 cargarResumenInicio();

```

7.4 Lógica del Formulario de Compra (formulario.js)

Se calcula el total a pagar iterando sobre el carrito y sumando precios.

```

1  const formularioPedido = document.getElementById("formulario-pedido");
2  const resumenCarritoPedido = document.getElementById("resumen-carrito-pedido");
3  const totalCarritoPedido = document.getElementById("total-carrito-pedido");
4  const tipoEntrega = document.getElementById("tipo_entrega");
5  const direccion = document.getElementById("direccion");
6  const tienda = document.getElementById("tienda");
7  const referencia = document.getElementById("referencia");
8
9  let carrito = JSON.parse(localStorage.getItem("carrito_chapatupromo")) || [];
10
11 function mostrarMensajePedido(texto, tipo) {
12   const mensaje = document.getElementById("mensaje-pedido");
13   mensaje.textContent = texto;
14
15   if (tipo === "error") {
16     mensaje.className = "mensaje-formulario mensaje-error";
17   } else {
18     mensaje.className = "mensaje-formulario mensaje-exito";
19   }
20 }
21
22 function calcularTotalCarrito() {
23   let total = 0;
24
25   carrito.forEach((producto) => {
26     total = total + (producto.precio * producto.cantidad);
27   });
28
29   return total;
30 }
31
32 function actualizarTotalPedido() {
33   const subtotal = calcularTotalCarrito();
34   let delivery = 0;
35
36   if (tipoEntrega.value === "delivery") {
37     delivery = 5;
38   }
39
40   totalCarritoPedido.textContent = subtotal.toFixed(2);
41   document.getElementById("total-final-pedido").textContent = (subtotal + delivery).toFixed(2);
42 }
43
main

```

Se dibujan los productos del carrito en una tabla resumen antes de pagar.

```

43 |
44 function mostrarCarritoPedido() {
45   if (carrito.length === 0) {
46     resumenCarritoPedido.innerHTML = `
47       <p>No tienes productos en el carrito.</p>
48       <p><a href="menu_chapatupromo.html">Volver al menu para agregar productos</a></p>
49     `;
50     actualizarTotalPedido();
51     formularioPedido.querySelector("button[type='submit']").disabled = true;
52     return;
53   }
54
55   let filas = "";
56
57   carrito.forEach((producto) => {
58     const subtotal = producto.precio * producto.cantidad;
59
60     filas += `
61       <tr>
62         <td>${producto.nombre}</td>
63         <td>${producto.cantidad}</td>
64         <td>S/. ${producto.precio.toFixed(2)}</td>
65         <td><strong>S/. ${subtotal.toFixed(2)}</strong></td>
66       </tr>
67     `;
68   });
69
70   resumenCarritoPedido.innerHTML = `
71     <table>
72       <tr>
73         <th class="celda-primer">Producto</th>
74         <th>Cantidad</th>
75         <th>Precio</th>
76         <th class="celda-ultima">Subtotal</th>
77       </tr>
78       ${filas}
79     </table>
80   `;
81
82   actualizarTotalPedido();
83   formularioPedido.querySelector("button[type='submit']").disabled = false;
84 }
85
main

```

Se valida que los datos personales y el método de pago no estén vacíos.

```

85
86 function validarFormularioPedido() {
87   const nombreCliente = document.getElementById("nombre_cliente").value.trim();
88   const telefonoCliente = document.getElementById("telefono").value.trim();
89   const correoCliente = document.getElementById("correo_cliente").value.trim();
90   const entrega = tipoEntrega.value;
91   const tiendaSeleccionada = tienda.value;
92   const metodoPago = document.getElementById("pago").value;
93
94   const soloLetras = /^[A-Za-zÁÉÍÓÚáéíóúÑñ ]+$/;
95   const correoValido = /^[A-Za-z0-9][A-Za-z0-9._-]*@[A-Za-z0-9-]+\.[A-Za-z]{2,}$/;
96   const celularValido = /^9[0-9]{8}$/;
97
98   if (nombreCliente === "" || telefonoCliente === "" || correoCliente === "") {
99     mostrarMensajePedido("Completa tus datos personales.", "error");
100    alert("Completa tus datos");
101    return false;
102  }
103
104   if (!soloLetras.test(nombreCliente)) {
105     mostrarMensajePedido("El nombre solo debe tener letras y espacios.", "error");
106     alert("Revisa el nombre del cliente");
107     return false;
108   }
109
110   if (!celularValido.test(telefonoCliente)) {
111     mostrarMensajePedido("El celular debe empezar en 9 y tener 9 digitos.", "error");
112     alert("Revisa el celular");
113     return false;
114   }
115
116   if (!correoValido.test(correoCliente)) {
117     mostrarMensajePedido("Ingresa un correo valido.", "error");
118     alert("Revisa el correo");
119     return false;
120   }
121
122   if (carrito.length === 0) {
123     mostrarMensajePedido("No puedes finalizar un pedido sin productos.", "error");
124     alert("Tu carrito esta vacio");
125     return false;
126   }
127
main

```

Se envía el paquete de datos del pedido al servidor vía fetch POST.

```

214 .catch((error) => {
215     mostrarMensajePedido("No se pudo conectar con el backend.", "error");
216     console.log(error);
217 });
218 }
219
220 function cambiarEntrega() {
221     if (tipoEntrega.value === "delivery") {
222         direccion.disabled = false;
223         referencia.disabled = false;
224         direccion.placeholder = "Ej: Jr. Las Flores 234, Miraflores";
225         referencia.placeholder = "Ej: Frente al parque, puerta azul";
226         tienda.value = "";
227         tienda.disabled = true;
228     } else if (tipoEntrega.value === "recojo") {
229         direccion.value = "";
230         referencia.value = "";
231         direccion.disabled = true;
232         referencia.disabled = true;
233         direccion.placeholder = "Solo necesario si eliges delivery";
234         referencia.placeholder = "Solo necesario si eliges delivery";
235     } else {
236         tienda.value = "";
237         direccion.value = "";
238         referencia.value = "";
239         tienda.disabled = true;
240         direccion.disabled = true;
241         referencia.disabled = true;
242         direccion.placeholder = "Elige primero el tipo de entrega";
243         referencia.placeholder = "Elige primero el tipo de entrega";
244     }
245 }
246
247 actualizarTotalPedido();
248 }
249
250 formularioPedido.addEventListener("submit", finalizarPedido);
251 tipoEntrega.addEventListener("change", cambiarEntrega);
252
253 llenarDatosUsuario();
254 mostrarCarritoPedido();
255 cambiarEntrega();

```


7.5 Lógica de Ofertas (ofertas.js)

Se implementan utilidades como mostrar mensajes y la función para agregar las ofertas al carrito local.

```

index.js x login.js x registrar.js x formulario.js x pedidos.js x ofe
1 const contenedorOfertas = document.getElementById("contenedor-ofertas");
2 let carrito = JSON.parse(localStorage.getItem("carrito_chapatupromo")) || [];
3
4 function mostrarMensajeOfertas(texto, tipo) {
5     const mensaje = document.getElementById("mensaje-ofertas");
6     mensaje.textContent = texto;
7
8     if (tipo === "error") {
9         mensaje.className = "mensaje-formulario mensaje-error";
10    } else {
11        mensaje.className = "mensaje-formulario mensaje-exito";
12    }
13 }
14
15 function guardarCarrito() {
16     localStorage.setItem("carrito_chapatupromo", JSON.stringify(carrito));
17 }
18
19 function agregarOfertaAlCarrito(id, nombre, precio, stock) {
20     const productoEncontrado = carrito.find((producto) => producto.id === id);
21
22     if (productoEncontrado) {
23         if (productoEncontrado.cantidad >= stock) {
24             mostrarMensajeOfertas("No puedes agregar mas unidades. Stock disponible: " + stock, "error");
25             alert("No hay mas stock disponible para este producto");
26             return;
27         }
28
29         productoEncontrado.cantidad = productoEncontrado.cantidad + 1;
30     } else {
31         carrito.push({
32             id: id,
33             nombre: nombre,
34             precio: Number(precio),
35             stock: Number(stock),
36             cantidad: 1
37         });
38     }
39
40     guardarCarrito();
41     mostrarMensajeOfertas("Producto agregado al carrito: " + nombre, "exito");
42     alert("Producto agregado al carrito");
43 }

```

Se inyectan las tarjetas de ofertas dinámicas usando innerHTML para mostrarlas en la tabla.

```

43 }
44
45 function actualizarResumenOfertas(ofertas) {
46   const hoy = ofertas.filter((producto) => producto.dias_para_vencer === 0).length;
47   const manana = ofertas.filter((producto) => producto.dias_para_vencer === 1).length;
48   const dosDias = ofertas.filter((producto) => producto.dias_para_vencer === 2).length;
49
50   document.getElementById("ofertas-hoy").textContent = hoy;
51   document.getElementById("ofertas-manana").textContent = manana;
52   document.getElementById("ofertas-dos-dias").textContent = dosDias;
53 }
54
55 function crearTablaOfertas(titulo, claseTitulo, claseTabla, productos) {
56   if (productos.length === 0) {
57     return "";
58   }
59
60   let filas = "";
61
62   productos.forEach((producto) => {
63     filas += `
64       <tr>
65         <td><strong>${producto.nombre}</strong></td>
66         <td>${producto.categoria}</td>
67         <td>${producto.descripcion}</td>
68         <td><del>$/. ${producto.precio_original}</del></td>
69         <td><span class="etiqueta-aviso etiqueta-roja">${producto.descuento_porcentaje}% OFF</span></td>
70         <td><strong>$/. ${producto.precio_chapatupromo}</strong></td>
71         <td>${producto.stock}</td>
72         <td>
73           <button type="button" onclick="agregarOfertaAlCarrito(${producto.id}, '${producto.nombre}', '${producto.precio_chapatupromo}', ${producto
74             .stock})">
75             Añadir al carrito
76           </button>
77         </td>
78       </tr>
79     `;
80   });
81
82   return `
83     <h3 class="titulo-seccion-ofertas ${claseTitulo}">${titulo}</h3>
84     <table class="lista-de-ofertas ${claseTabla}">
85       <tr>

```

Se renderizan independientemente las ofertas que vencen hoy, mañana o pasado filtrándolas por sus días.

```

86     <tr>
87       <td colspan="7">${filas}</td>
88     </tr>
89   </table>
90 `;
91 }
92
93 function mostrarOfertas(ofertas) {
94   actualizarResumenOfertas(ofertas);
95
96   const ofertasHoy = ofertas.filter((producto) => producto.dias_para_vencer === 0);
97   const ofertasManana = ofertas.filter((producto) => producto.dias_para_vencer === 1);
98   const ofertasDosDias = ofertas.filter((producto) => producto.dias_para_vencer === 2);
99   const ofertasTresDias = ofertas.filter((producto) => producto.dias_para_vencer === 3);
100
101   let contenido = "";
102   contenido += crearTablaOfertas('<i class="fas fa-fire"></i> VENCEN HOY - 50% de descuento', "rojo-hoy", "tabla-vence-hoy", ofertasHoy);
103   contenido += crearTablaOfertas('<i class="fas fa-clock"></i> VENCEN MANANA - 40% de descuento', "naranja-manana", "tabla-vence-manana", ofertasMa
104     nana);
105   contenido += crearTablaOfertas('<i class="fas fa-calendar-day"></i> VENCEN EN 2 DIAS - 30% de descuento', "purpura-dos-dias", "tabla-vence-pasado
106     ", ofertasDosDias);
107   contenido += crearTablaOfertas('<i class="fas fa-calendar-check"></i> VENCEN EN 3 DIAS - 20% de descuento', "purpura-dos-dias", "tabla-vence-pasa
108     do", ofertasTresDias);
109
110   if (contenido === "") {
111     contenedorOfertas.innerHTML = "<p>No hay ofertas disponibles por ahora.</p>";
112   } else {
113     contenedorOfertas.innerHTML = contenido;
114   }
115 }
116
117 function cargarOfertas() {
118   fetch(API_URL + "/api/ofertas")
119     .then((respuesta) => respuesta.json())
120     .then((ofertas) => {
121       mostrarOfertas(ofertas);
122     })
123     .catch((error) => {
124       mostrarMensajeOfertas("No se pudieron cargar las ofertas.", "error");
125       console.log(error);
126     });
127 }
128
129 cargarOfertas();

```

7.6 Lógica de Pedidos Realizados (pedidos.js)

Se formatea la fecha del pedido para que sea legible por el usuario, además de otras utilidades de estado.

```

1 const tablaPedidos = document.getElementById("tabla-pedidos");
2 let pedidosActuales = [];
3 let intervaloEstados = null;
4
5 function mostrarMensajePedidos(texto, tipo) {
6   const mensaje = document.getElementById("mensaje-pedidos");
7   mensaje.textContent = texto;
8
9   if (tipo === "error") {
10    mensaje.className = "mensaje-formulario mensaje-error";
11   } else {
12    mensaje.className = "mensaje-formulario mensaje-exito";
13   }
14 }
15
16 function formatearFecha(fechaTexto) {
17   const fecha = new Date(fechaTexto);
18   return fecha.toLocaleDateString("es-PE") + " " + fecha.toLocaleTimeString("es-PE");
19 }
20
21 function esDelivery(pedido) {
22   return pedido.tipo_entrega === "delivery";
23 }
24
25 function obtenerEstados(pedido) {
26   if (esDelivery(pedido)) {
27     return ["Pendiente", "Preparando", "En camino", "Entregado"];
28   }
29
30   return ["Pendiente", "Preparando", "Listo para recojo", "Recogido"];
31 }
32
33 function obtenerEstadoActual(pedido) {
34   if (!esDelivery(pedido) && pedido.estado === "Entregado") {
35     return "Recogido";
36   }
37
38   if (esDelivery(pedido) && pedido.estado === "Listo para recojo") {
39     return "En camino";
40   }
41
42   return pedido.estado;
43 }

```

Se implementan funciones para calcular el total del pedido y actualizar las estadísticas superiores (.filter()).

```

40 }
41
42 return pedido.estado;
43 }
44
45 function pedidoFinalizado(pedido) {
46   const estado = obtenerEstadoActual(pedido);
47   return estado === "Entregado" || estado === "Recogido";
48 }
49
50 function calcularTotalPedido(pedido) {
51   let total = Number(pedido.total);
52
53   if (esDelivery(pedido)) {
54     total = total + 5;
55   }
56
57   return total.toFixed(2);
58 }
59
60 function mostrarEntrega(pedido) {
61   if (esDelivery(pedido)) {
62     return "Delivery a domicilio";
63   }
64   return "Recojo en tienda - " + pedido.tienda;
65 }
66
67
68 function actualizarEstadisticas(pedidos) {
69   const completados = pedidos.filter((pedido) => pedidoFinalizado(pedido)).length;
70   const proceso = pedidos.length - completados;
71
72   document.getElementById("total-pedidos").textContent = pedidos.length;
73   document.getElementById("pedidos-listos").textContent = completados;
74   document.getElementById("pedidos-proceso").textContent = proceso;
75 }
76
77 function mostrarPedidos(pedidos) {
78   if (pedidos.length === 0) {
79     tablaPedidos.innerHTML = "<p>Todavía no hay pedidos registrados.</p>";
80     actualizarEstadisticas(pedidos);
81     return;
82   }

```

Se inyecta la fila de la tabla HTML con un menú select para actualizar el estado del pedido o cancelarlo.

```

82 }
83
84 let filas = "";
85
86 pedidos.forEach((pedido) => {
87   const estado = obtenerEstadoActual(pedido);
88
89   filas += `
90     <tr>
91       <td>${pedido.id}</td>
92       <td>${formatearFecha(pedido.fecha_pedido)}</td>
93       <td>${pedido.nombre_cliente}</td>
94       <td>${pedido.productos}</td>
95       <td>S/. ${calcularTotalPedido(pedido)}</td>
96       <td>${mostrarEntrega(pedido)}</td>
97       <td>${pedido.metodo_pago}</td>
98       <td><span class="estado-pedido">${estado}</span></td>
99     </tr>
100   `;
101 });
102
103 tablaPedidos.innerHTML = `
104   <table>
105     <tr>
106       <th class="celda-primer">Pedido</th>
107       <th>Fecha</th>
108       <th>Cliente</th>
109       <th>Productos</th>
110       <th>Total</th>
111       <th>Entrega</th>
112       <th>Pago</th>
113       <th class="celda-ultima">Estado</th>
114     </tr>
115     ${filas}
116   </table>
117 `;
118
119 actualizarEstadisticas(pedidos);
120 }
121
122 function guardarEstadoPedido(id, estado) {
123   fetch(API_URL + "/api/pedidos/" + id + "/estado", {
124     method: "PUT",

```

Se ejecuta la consulta a la BD mediante fetch y se automatiza el avance de estados temporal.

```

142 function avanzarEstadosAutomaticamente() {
143   pedidosActuales.forEach((pedido) => {
144     const estados = obtenerEstados(pedido);
145     const estadoActual = obtenerEstadoActual(pedido);
146     const posicion = estados.indexOf(estadoActual);
147
148     if (posicion >= 0 && posicion < estados.length - 1) {
149       guardarEstadoPedido(pedido.id, estados[posicion + 1]);
150     }
151   });
152 }
153
154 function cargarPedidos() {
155   const usuario = JSON.parse(localStorage.getItem("usuario_chapatupromo"));
156
157   if (!usuario) {
158     tablaPedidos.innerHTML = "<p>Debes iniciar sesion para ver tus pedidos.</p>";
159     actualizarEstadisticas([]);
160     return;
161   }
162
163   fetch(API_URL + "/api/pedidos")
164     .then((respuesta) => respuesta.json())
165     .then((pedidos) => {
166       // Filtrar los pedidos para que solo vea los suyos
167       const misPedidos = pedidos.filter(p => p.correo_cliente === usuario.correo);
168
169       pedidosActuales = misPedidos;
170       mostrarPedidos(misPedidos);
171
172       if (!intervaloEstados) {
173         intervaloEstados = setInterval(avanzarEstadosAutomaticamente, 5000);
174       }
175     })
176     .catch((error) => {
177       mostrarMensajePedidos("No se pudieron cargar los pedidos.", "error");
178       console.log(error);
179     });
180 }
181
182 cargarPedidos();

```

7.7 Lógica de Login (login.js)

Se captura el correo y la contraseña, y se envía hacia el backend para validarlos (fetch POST).

```

1  const formularioLogin = document.getElementById("formulario-login");
2
3  function mostrarMensajeLogin(texto, tipo) {
4      const mensaje = document.getElementById("mensaje-login");
5      mensaje.textContent = texto;
6
7      if (tipo === "error") {
8          mensaje.className = "mensaje-formulario mensaje-error";
9      } else {
10         mensaje.className = "mensaje-formulario mensaje-exito";
11     }
12 }
13
14 function iniciarSesion(evento) {
15     evento.preventDefault();
16
17     const correo = document.getElementById("usuario").value.trim();
18     const password = document.getElementById("contrasena").value;
19     const correoValido = /^[A-Za-z0-9][A-Za-z0-9._-]*@[A-Za-z0-9]+\.[A-Za-z]{2,}$/;
20
21     if (correo.length === 0 || password.length === 0) {
22         mostrarMensajeLogin("Completa tu correo y contrasena.", "error");
23         alert("Completa todos los campos");
24         return;
25     }
26
27     if (!correoValido.test(correo)) {
28         mostrarMensajeLogin("Ingresa un correo valido.", "error");
29         alert("Correo invalido");
30         return;
31     }
32
33     fetch(API_URL + "/api/login", {
34         method: "POST",
35         headers: {
36             "Content-Type": "application/json"
37         },
38         body: JSON.stringify({
39             correo: correo,
40             password: password
41         })
42     })
43     .then((respuesta) => respuesta.json())

```

Si la BD responde éxito, se guarda el usuario en localStorage y se añade el evento al formulario.

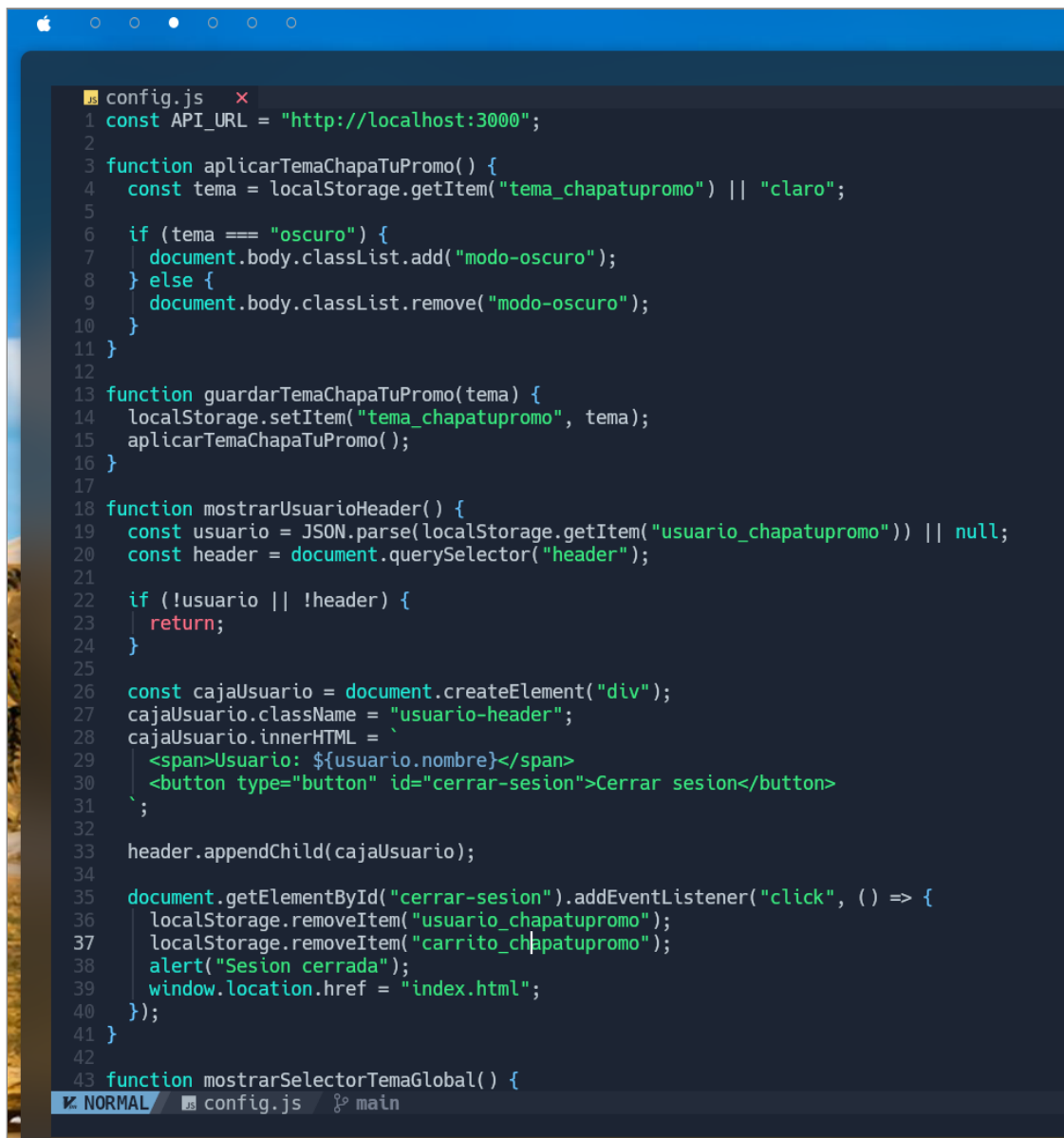
```

28  mostrarMensajeLogin("Ingresa un correo valido.", "error");
29  alert("Correo invalido");
30  return;
31  }
32
33  fetch(API_URL + "/api/login", {
34    method: "POST",
35    headers: {
36      "Content-Type": "application/json"
37    },
38    body: JSON.stringify({
39      correo: correo,
40      password: password
41    })
42  })
43  .then((respuesta) => respuesta.json())
44  .then((datos) => {
45    if (datos.usuario) {
46      localStorage.setItem("usuario_chapatupromo", JSON.stringify(datos.usuario));
47      localStorage.removeItem("carrito_chapatupromo");
48      mostrarMensajeLogin("Inicio de sesion correcto. Bienvenido " + datos.usuario.nombre, "exito");
49      alert("Inicio de sesion correcto");
50      window.location.href = "index.html";
51    } else {
52      mostrarMensajeLogin(datos.mensaje || "Correo o contrasena incorrectos.", "error");
53    }
54  })
55  .catch((error) => {
56    mostrarMensajeLogin("No se pudo conectar con el backend.", "error");
57    console.log(error);
58  });
59  }
60
61  formularioLogin.addEventListener("submit", iniciarSesion);
62
63

```


7.8 Lógica de Configuración Global (config.js)

Se añade una lógica global para leer si hay sesión abierta y verificar el tema seleccionado.



```

1  config.js
2  const API_URL = "http://localhost:3000";
3
4  function aplicarTemaChapaTuPromo() {
5      const tema = localStorage.getItem("tema_chapatupromo") || "claro";
6
7      if (tema === "oscuro") {
8          document.body.classList.add("modo-oscuro");
9      } else {
10         document.body.classList.remove("modo-oscuro");
11     }
12 }
13
14 function guardarTemaChapaTuPromo(tema) {
15     localStorage.setItem("tema_chapatupromo", tema);
16     aplicarTemaChapaTuPromo();
17 }
18
19 function mostrarUsuarioHeader() {
20     const usuario = JSON.parse(localStorage.getItem("usuario_chapatupromo")) || null;
21     const header = document.querySelector("header");
22
23     if (!usuario || !header) {
24         return;
25     }
26
27     const cajaUsuario = document.createElement("div");
28     cajaUsuario.className = "usuario-header";
29     cajaUsuario.innerHTML = `
30         <span>Usuario: ${usuario.nombre}</span>
31         <button type="button" id="cerrar-sesion">Cerrar sesion</button>
32     `;
33     header.appendChild(cajaUsuario);
34
35     document.getElementById("cerrar-sesion").addEventListener("click", () => {
36         localStorage.removeItem("usuario_chapatupromo");
37         localStorage.removeItem("carrito_chapatupromo");
38         alert("Sesion cerrada");
39         window.location.href = "index.html";
40     });
41 }
42
43 function mostrarSelectorTemaGlobal() {

```

Se inyecta el selector de temas para alternar entre oscuro y claro de forma global.

```

37   localStorage.removeItem("carrito_chapatupromo");
38   alert("Sesion cerrada");
39   window.location.href = "index.html";
40   });
41 }
42
43 function mostrarSelectorTemaGlobal() {
44   const nav = document.querySelector("nav");
45   if (!nav) return;
46
47   const divControl = document.createElement("div");
48   divControl.style.marginLeft = "auto";
49   divControl.style.display = "flex";
50   divControl.style.alignItems = "center";
51   divControl.style.gap = "10px";
52
53   divControl.innerHTML = `
54     <label for="tema-global" style="color: var(--blanco-puro); font-weight: 600; font-size: 14px;">Apariencia:</label>
55     <select id="tema-global" style="padding: 4px; border-radius: 4px; border: none;">
56       <option value="claro">Modo Claro</option>
57       <option value="oscuro">Modo Oscuro</option>
58     </select>
59   `;
60
61   nav.appendChild(divControl);
62
63   const selectTema = document.getElementById("tema-global");
64   const temaActual = localStorage.getItem("tema_chapatupromo") || "claro";
65   selectTema.value = temaActual;
66
67   selectTema.addEventListener("change", (e) => {
68     guardarTemaChapaTuPromo(e.target.value);
69   });
70 }
71
72 aplicarTemaChapaTuPromo();
73 mostrarUsuarioHeader();
74 mostrarSelectorTemaGlobal();

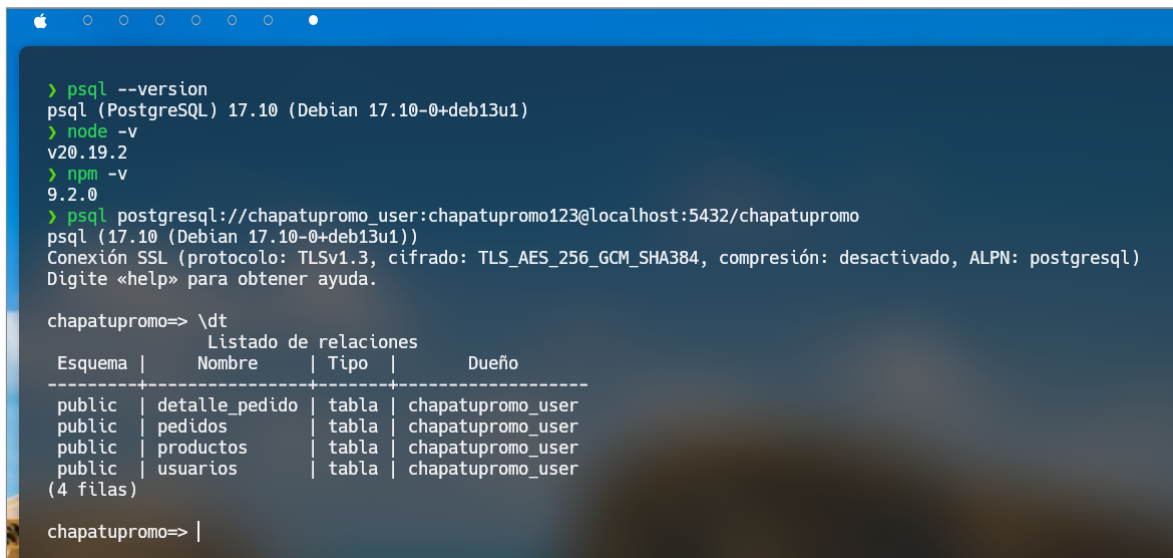
```

8. CAPTURAS DE BASE DE DATOS Y BACKEND (NODE.JS)

Estas capturas demuestran la estructura relacional creada en PostgreSQL y la correcta ejecución de consultas SQL a través del entorno de comandos.

8.1 Estructura de Tablas (Relacional)

Se evidencian las 4 tablas principales creadas para el sistema (usuarios, productos, pedidos y detalle_pedido) usando el comando \dt.



```

> psql --version
psql (PostgreSQL) 17.10 (Debian 17.10-0+deb13u1)
> node -v
v20.19.2
> npm -v
9.2.0
> psql postgresql://chapatupromo_user:chapatupromo123@localhost:5432/chapatupromo
psql (17.10 (Debian 17.10-0+deb13u1))
Conexión SSL (protocolo: TLSv1.3, cifrado: TLS_AES_256_GCM_SHA384, compresión: desactivado, ALPN: postgresql)
Digite «help» para obtener ayuda.

chapatupromo=> \dt
          Listado de relaciones
-----
Esquema | Nombre | Tipo | Dueño
-----
public  | detalle_pedido | tabla | chapatupromo_user
public  | pedidos      | tabla | chapatupromo_user
public  | productos    | tabla | chapatupromo_user
public  | usuarios     | tabla | chapatupromo_user
(4 filas)

chapatupromo=> |

```

8.2 Consulta de Selección (SELECT)

Se demuestra una consulta SELECT trayendo los productos activos de la base de datos para verificar la persistencia de los datos.

```
> psql --version
psql (PostgreSQL) 17.10 (Debian 17.10-0+deb13u1)
> node -v
v20.19.2
> npm -v
9.2.0
> psql postgresql://chapatupromo_user:chapatupromo123@localhost:5432/chapatupromo
psql (17.10 (Debian 17.10-0+deb13u1))
Conexión SSL (protocolo: TLSv1.3, cifrado: TLS_AES_256_GCM_SHA384, compresión: desactivado, ALPN: postgresql)
Digite «help» para obtener ayuda.

chapatupromo=> \dt
Listado de relaciones
Esquema | Nombre | Tipo | Dueño
-----+-----+-----+-----
public | detalle_pedido | tabla | chapatupromo_user
public | pedidos | tabla | chapatupromo_user
public | productos | tabla | chapatupromo_user
public | usuarios | tabla | chapatupromo_user
(4 filas)

chapatupromo=> SELECT id, nombre, categoria, precio_original, stock FROM productos LIMIT 5;
 id | nombre | categoria | precio_original | stock
-----+-----+-----+-----+-----
 2 | Torta Tres Leches 800g | Repostería y Panadería | 38.00 | 4
 3 | Queque Marmoleado Donofrio 500g | Repostería y Panadería | 14.00 | 6
 4 | Pan de Molde Bimbo Integral 500g | Repostería y Panadería | 6.00 | 10
 5 | Yogurt Gloria Fresa 1L | Lacteos | 8.50 | 5
 6 | Queso Fresco La Preferida 250g | Lacteos | 9.00 | 6
(5 filas)

chapatupromo=> |
```

8.3 Inserción y Actualización (INSERT & UPDATE)

Se ejecuta un INSERT INTO para crear un nuevo usuario y posteriormente un UPDATE para modificar su nombre, demostrando el uso de sentencias DML.

```
> psql --version
psql (PostgreSQL) 17.10 (Debian 17.10-0+deb13u1)
> node -v
v20.19.2
> npm -v
9.2.0
> psql postgresql://chapatupromo_user:chapatupromo123@localhost:5432/chapatupromo
psql (17.10 (Debian 17.10-0+deb13u1))
Conexión SSL (protocolo: TLSv1.3, cifrado: TLS_AES_256_GCM_SHA384, compresión: desactivado, ALPN: postgresql)
Digite «help» para obtener ayuda.

chapatupromo=> \dt
Listado de relaciones
Esquema | Nombre | Tipo | Dueño
-----+-----+-----+-----
public | detalle_pedido | tabla | chapatupromo_user
public | pedidos | tabla | chapatupromo_user
public | productos | tabla | chapatupromo_user
public | usuarios | tabla | chapatupromo_user
(4 filas)

chapatupromo=> SELECT id, nombre, categoria, precio_original, stock FROM productos LIMIT 5;
 id | nombre | categoria | precio_original | stock
-----+-----+-----+-----+-----
 2 | Torta Tres Leches 800g | Repostería y Panadería | 38.00 | 4
 3 | Queque Marmoleado Donofrio 500g | Repostería y Panadería | 14.00 | 6
 4 | Pan de Molde Bimbo Integral 500g | Repostería y Panadería | 6.00 | 10
 5 | Yogurt Gloria Fresa 1L | Lacteos | 8.50 | 5
 6 | Queso Fresco La Preferida 250g | Lacteos | 9.00 | 6
(5 filas)

chapatupromo=> INSERT INTO usuarios (nombre, correo, password, telefono, direccion) VALUES ('Jesamin Zevallos', 'jzevallos@utp.edu.pe', 'seguro123', '987654321', 'Sede UTP');
INSERT 0 1
chapatupromo=> UPDATE usuarios SET nombre = 'M.Sc. Jesamin Melissa Zevallos Quispe' WHERE correo = 'jzevallos@utp.edu.pe';
UPDATE 1
chapatupromo=> SELECT id, nombre, correo FROM usuarios WHERE correo = 'jzevallos@utp.edu.pe';
 id | nombre | correo
-----+-----+-----
 5 | M.Sc. Jesamin Melissa Zevallos Quispe | jzevallos@utp.edu.pe
(1 fila)

chapatupromo=> |
```

9. LINK DEL REPOSITORIO

El código fuente completo del proyecto se encuentra disponible en GitHub:

[https://github.com/MaxBenitesC/Study-UTP-System/tree/main/CICLO_V/
TALLER_DE_PROGRAMACION_WEB/SEMANA_15](https://github.com/MaxBenitesC/Study-UTP-System/tree/main/CICLO_V/TALLER_DE_PROGRAMACION_WEB/SEMANA_15)

10. OBSERVACIONES Y CONCLUSIONES

El desarrollo del Avance de Proyecto Final 3 evidencia la aplicación práctica de los conocimientos adquiridos a lo largo del curso de Taller de Programación Web:

1. **Implementación de Lógica Frontend:** Se logró integrar interactividad en la página web mediante el uso de JavaScript (Vanilla JS), permitiendo la manipulación del DOM y la gestión de eventos, cumpliendo con los objetivos de dinamismo requeridos para el ciclo.
2. **Integración con Base de Datos:** Se demostró la correcta conexión de la interfaz web con un sistema de gestión de base de datos relacional (PostgreSQL), aplicando consultas fundamentales (SELECT, INSERT, UPDATE) para asegurar la persistencia de la información del proyecto.
3. **Consolidación de Competencias:** El proyecto unifica los fundamentos de estructuración (HTML5), diseño visual (CSS3) y programación lógica, evidenciando el dominio de las herramientas necesarias para la construcción de aplicaciones web funcionales en un entorno académico.